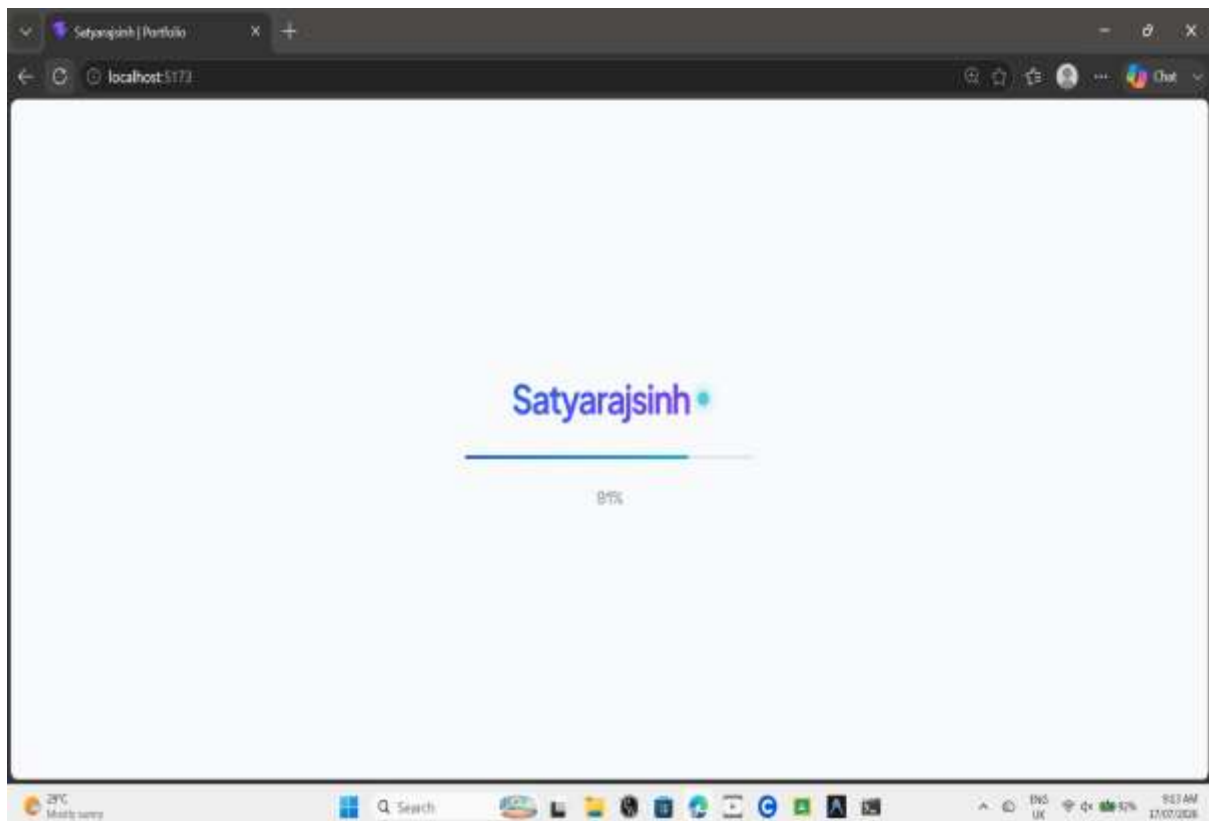


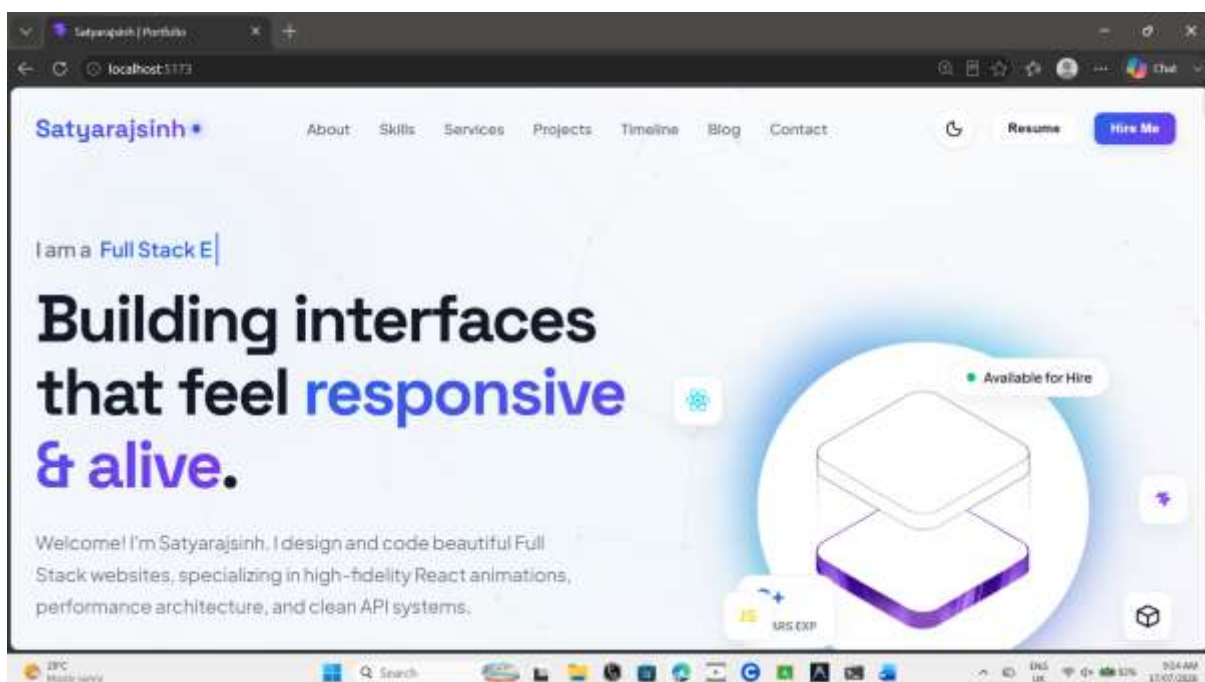
PRACTICAL-1

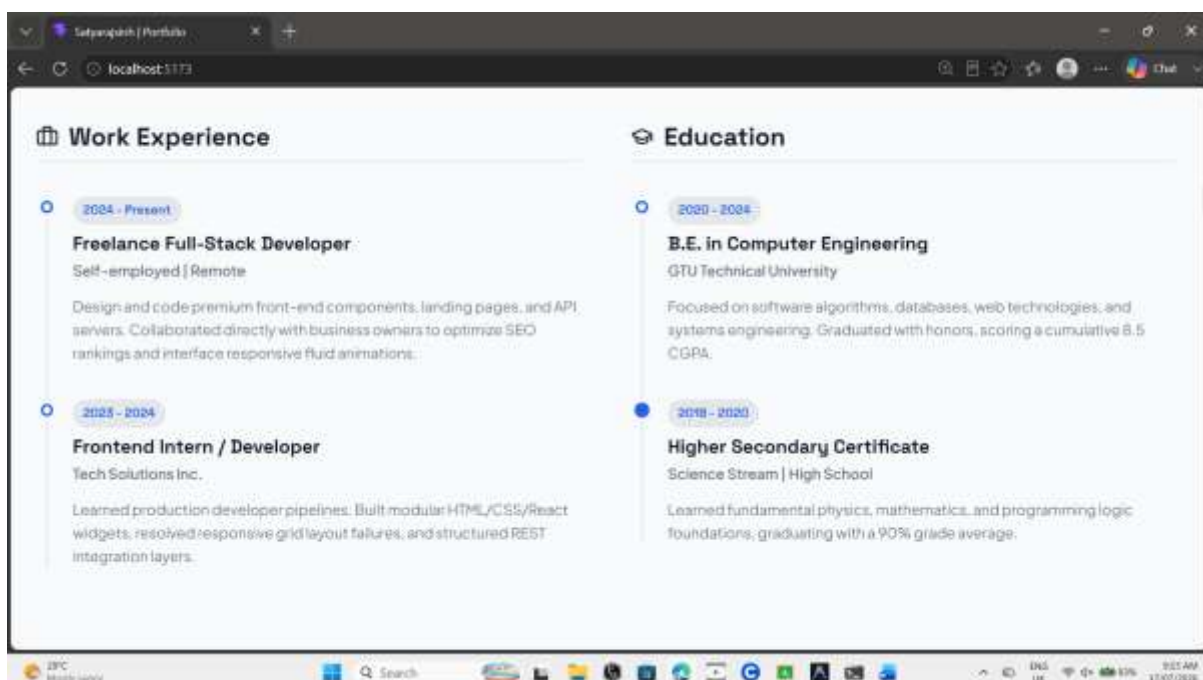
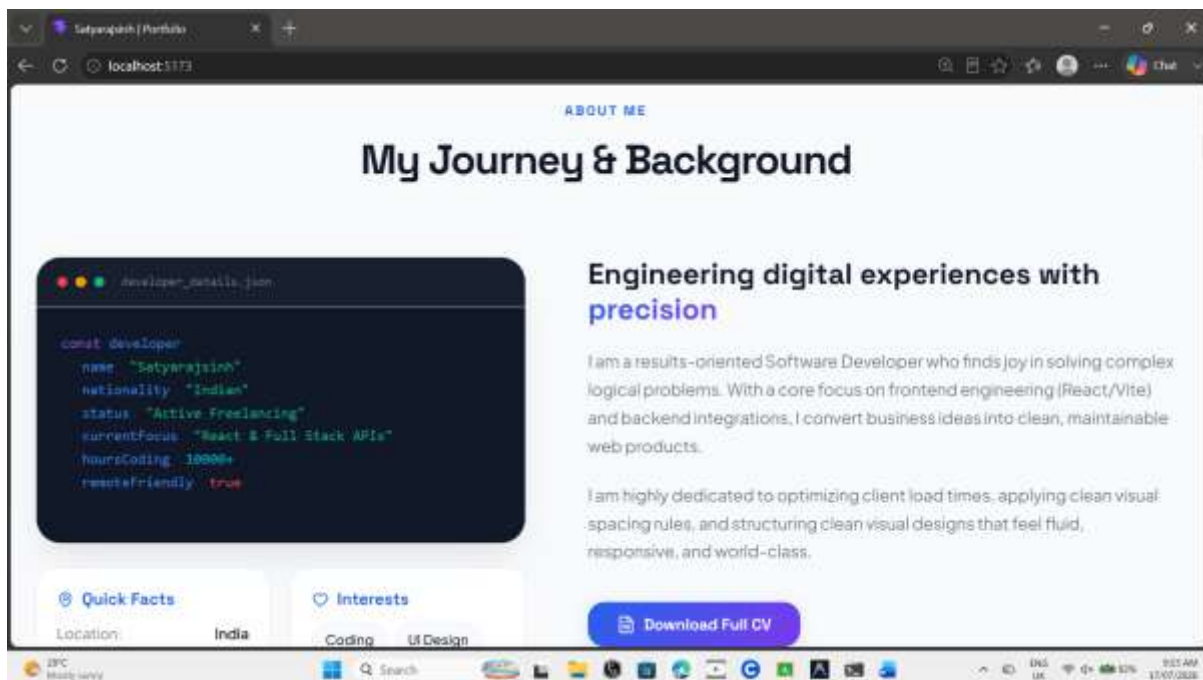
Problem Defination:

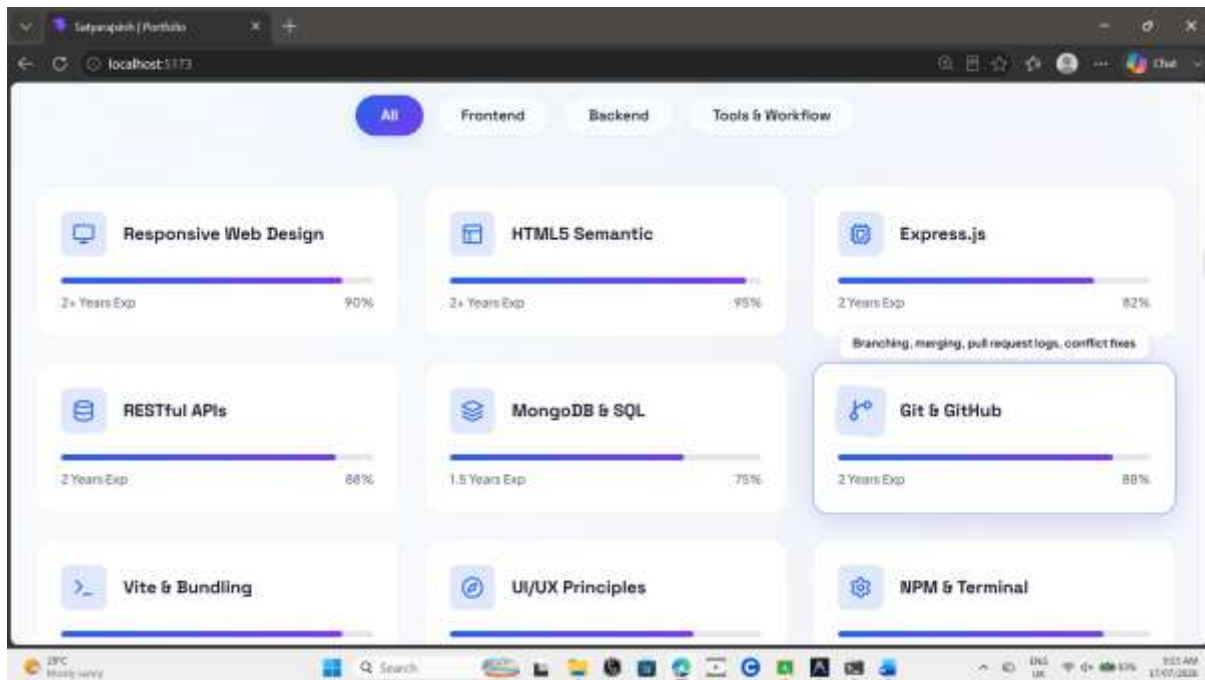
- Create a React application using Vite for a student portfolio page.
- The application must include at least 4 reusable components: Header,About, Skills, and Footer.
- Each component must be independently structured and composed into a single-page layout.
- No logic or JSX should be duplicated across components.
- Props must be used to pass at least one piece of data into a minimum of 2 components.

OUTPUT:









Key-Questions:

- Question:** What is the role of each component in the overall UI structure?

Answer: Each React component represents a specific part of the user interface, such as the Header, About, Skills, or Footer. Components help organize the application into small, independent, and reusable sections. This modular structure improves code readability, simplifies maintenance, and makes the application easier to scale and update.
- Question:** How does React re-render when props change?

Answer: When a parent component passes new props to a child component, React automatically re-renders the child component. It first compares the new Virtual DOM with the previous Virtual DOM to detect changes. Then, React updates only the affected elements in the real DOM, making the rendering process efficient and improving application performance.
- Question:** Why is component reusability important in large-scale applications?

Answer: Component reusability allows developers to build a component once and use it multiple times throughout the application. This reduces code duplication, ensures a consistent user interface, and speeds up development. It also makes debugging, testing, and maintaining large-scale applications much easier because changes only need to be made in one place.

Conclusion: This practical demonstrated how to build a React application using Vite with a reusable component-based architecture. Separate components such as Header, About, Skills, and Footer were created and combined into a single-page portfolio. The use of props enabled data sharing between components, resulting in a modular, maintainable, and efficient React application.